

Learning When to Adapt

DISeL — Dynamic Input-Sensitive LoRA

Ali Zindari^{1,2} Xiaowen Jiang^{1,2} Rotem Mulayoff² Sebastian U. Stich²

¹Universität des Saarlandes · ²CISPA Helmholtz Center for Information Security, Saarbrücken, Germany

CODE github.com/alizindari/DISeL · CONTACT {ali.zindari, xiaowen.jiang, rotem.mulayoff, stich}@cispa.de

TL;DR LoRA applies the *same* low-rank correction to *every* input. This forces an inherent compromise between adapting to the task and preserving pre-trained behavior. **DISeL** places lightweight input-dependent sigmoid gates on individual rank-one components, so the adapter is **active only when needed**. Same low-rank structure, near-zero added cost, and **retention close to the base model** across LLaMA, Mistral, and RoBERTa.

1 The forgetting problem

A fine-tuned model adds a learnable correction $f(\mathbf{x}; \theta)$ on top of the frozen pre-trained weights:

$$\mathbf{y} = \mathbf{W}_0 \mathbf{x} + f(\mathbf{x}; \theta).$$

Full FT, LoRA, DoRA, AdaLoRA are all the special case $f(\mathbf{x}; \theta) = \Delta \mathbf{x}$, with a *fixed* matrix Δ that does *not* depend on $\mathbf{x}$. They differ only in how Δ is parameterized.

The same correction $\Delta \mathbf{x}$ is therefore applied to fine-tuning inputs (where adaptation is needed) *and* to pre-training inputs (where the original mapping should be preserved).

Consequence: task accuracy improves, but the model loses general capability. This is *catastrophic forgetting*.

Question. Is there a structural reason a fixed Δ must forget, even in the simplest possible setting?

2 A minimal linear analysis

Symmetric mixture of fine-tuning and pre-training populations:

$$\mathbf{x} \sim \begin{cases} p_{\text{ft}} & \text{w.p. } \frac{1}{2}, \\ p_{\text{pt}} & \text{w.p. } \frac{1}{2}, \end{cases} \quad \mathbf{y} = \begin{cases} (\mathbf{W}_0 + \mathbf{M}) \mathbf{x} & \text{when } \mathbf{x} \sim p_{\text{ft}}, \\ \mathbf{W}_0 \mathbf{x} & \text{when } \mathbf{x} \sim p_{\text{pt}}. \end{cases}$$

Loss after subtracting the frozen mapping:

$$\mathcal{L}(f) = \frac{1}{2} \mathbb{E}_{p_{\text{ft}}} [\|f(\mathbf{x}) - \mathbf{M} \mathbf{x}\|^2] + \frac{1}{2} \mathbb{E}_{p_{\text{pt}}} [\|f(\mathbf{x})\|^2].$$

No fixed correction can do both

With equal second moments Σ , the optimal *fixed* Δ is

$$\Delta^* = \frac{1}{2} \mathbf{M}, \quad \mathcal{L}(\Delta^*) = \frac{1}{4} \text{Tr}(\mathbf{M} \Sigma \mathbf{M}^\top) > 0.$$

No fixed Δ , including full FT, can both fit the FT signal *and* preserve pre-training: it applies only *half* of the desired update on p_{ft} and a non-zero spurious update on p_{pt} .

Bayes-optimal input-dependent correction

Allowing f to depend on $\mathbf{x}$, the pointwise minimizer is

$$f^*(\mathbf{x}) = \pi_{\text{ft}}(\mathbf{x}) \mathbf{M} \mathbf{x}, \quad \pi_{\text{ft}}(\mathbf{x}) = \frac{p_{\text{ft}}(\mathbf{x})}{p_{\text{ft}}(\mathbf{x}) + p_{\text{pt}}(\mathbf{x})} \in [0, 1],$$

$$\mathcal{L}_{\text{Bayes}} = \frac{1}{2} \int \frac{p_{\text{ft}}(\mathbf{x}) p_{\text{pt}}(\mathbf{x})}{p_{\text{ft}}(\mathbf{x}) + p_{\text{pt}}(\mathbf{x})} \|\mathbf{M} \mathbf{x}\|^2 d\mathbf{x}.$$

A *soft switch*: $\pi_{\text{ft}} \rightarrow 1$ on FT inputs, $\pi_{\text{ft}} \rightarrow 0$ on pre-training inputs. No measurable correction can do better.

Gaussian case: the sigmoid is exact

When $p_{\text{ft}} = \mathcal{N}(\mu_{\text{ft}}, \Sigma)$ and $p_{\text{pt}} = \mathcal{N}(\mu_{\text{pt}}, \Sigma)$ share covariance, the log-density ratio is *affine*, so the posterior is a sigmoid:

$$\log \frac{p_{\text{ft}}(\mathbf{x})}{p_{\text{pt}}(\mathbf{x})} = \mathbf{w}_g^\top \mathbf{x} + b_g, \quad \pi_{\text{ft}}(\mathbf{x}) = \sigma(\mathbf{w}_g^\top \mathbf{x} + b_g),$$

$$\mathbf{w}_g = \Sigma^{-1}(\mu_{\text{ft}} - \mu_{\text{pt}}) \in \mathbb{R}^d, \quad b_g = \frac{1}{2}(\mu_{\text{pt}}^\top \Sigma^{-1} \mu_{\text{pt}} - \mu_{\text{ft}}^\top \Sigma^{-1} \mu_{\text{ft}}) \in \mathbb{R}.$$

Hence the Bayes-optimal correction has the closed form

$$f^*(\mathbf{x}) = \sigma(\mathbf{w}_g^\top \mathbf{x} + b_g) \mathbf{M} \mathbf{x}.$$

DISeL realizes f^* exactly

Factor $\mathbf{M} = \mathbf{A} \mathbf{B}$ (which exists whenever $\text{rank}(\mathbf{M}) \leq r$). Set the gate parameters

$$\mathbf{W}_g = \mathbf{1}_r \mathbf{w}_g^\top \in \mathbb{R}^{r \times d_g}, \quad \mathbf{b}_g = b_g \mathbf{1}_r \in \mathbb{R}^r.$$

Then every rank's gate carries the same value $g_i(\mathbf{x}) = \pi_{\text{ft}}(\mathbf{x})$, so

$$\mathbf{A} \mathbf{G}(\mathbf{x}) \mathbf{B} \mathbf{x} = \pi_{\text{ft}}(\mathbf{x}) \mathbf{A} \mathbf{B} \mathbf{x} = \pi_{\text{ft}}(\mathbf{x}) \mathbf{M} \mathbf{x} = f^*(\mathbf{x}).$$

A sigmoid-gated LoRA with the right parameters realizes the Bayes-optimal correction exactly. DISeL is the smallest learnable family containing this optimum.

3 Toy experiment

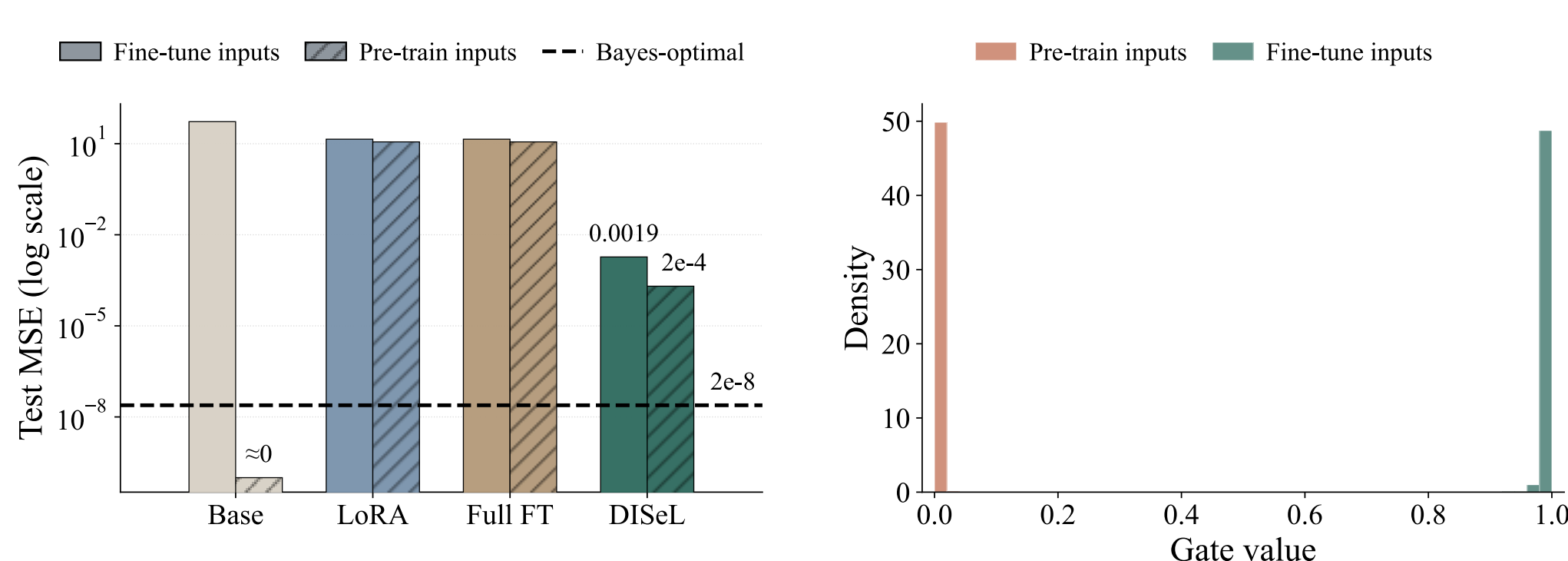


Fig. 1. Left: LoRA and Full FT hit the fixed-correction floor; DISeL approaches the Bayes-optimal error on both domains. Right: learned gates concentrate near 1 on FT inputs and near 0 on pre-training inputs.

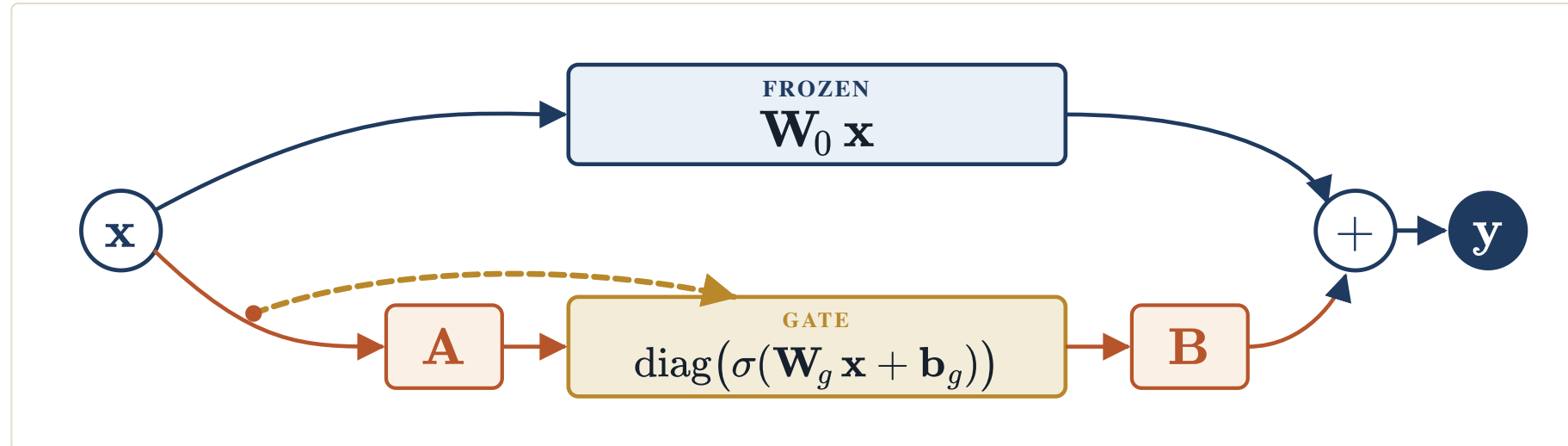
4 DISeL: input-dependent gates over rank-1 components

We keep LoRA's low-rank structure $\mathbf{A} \mathbf{B} \mathbf{x}$ with $\mathbf{A} \in \mathbb{R}^{d_y \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times d_x}$, but multiply each rank-one component by a learnable input-dependent gate $g_i(\mathbf{x}) \in [0, 1]$:

$$f(\mathbf{x}; \theta) = \mathbf{A} \mathbf{G}(\mathbf{x}) \mathbf{B} \mathbf{x} = \sum_{i=1}^r g_i(\mathbf{x}) \mathbf{a}_i \mathbf{b}_i^\top \mathbf{x},$$

$$\mathbf{G}(\mathbf{x}) = \text{diag}(g(\mathbf{x})), \quad g(\mathbf{x}) = \sigma(\mathbf{W}_g \mathbf{x} + \mathbf{b}_g) \in [0, 1]^r.$$

DISeL block



Design choices

- Init.** $\mathbf{b}_g = -3 \cdot \mathbf{1}$, so $\sigma(\mathbf{b}_g) \approx 0.05$. DISeL starts equal to the pre-trained model.
- Sigmoid (not softmax).** Rank-one components are independent soft switches.
- Two-LR.** Gates use a learning rate $5 \times$ larger than $(\mathbf{A}, \mathbf{B})$, since they solve a *routing* task, not a fitting one.

5 LLaMA & Mistral on math & code

LLaMA-2-7B on *MetaMathQA*; Mistral-7B on *Magicoder-Evol-Instruct*. Retention = mean accuracy on 14 OOD benchmarks (commonsense, world knowledge, medical QA, ...).

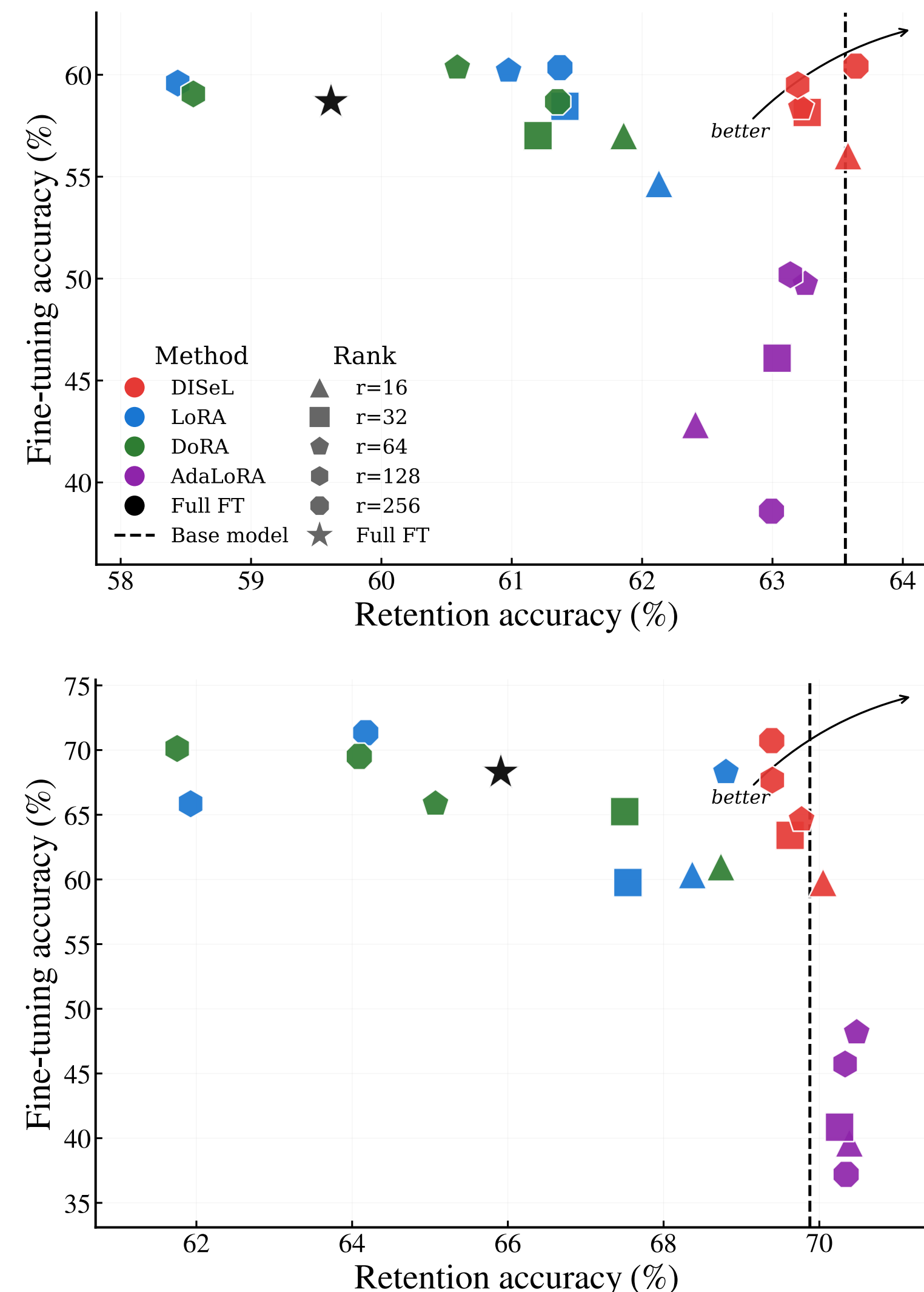


Fig. 2. FT accuracy vs. retention for LLaMA / math (top) and Mistral / code (bottom). DISeL stays near the base across $r \in \{16, 32, 64, 128, 256\}$.

- DISeL:** $\approx 60\%$ math accuracy (LLaMA) / $> 70\%$ code (Mistral) with retention within a fraction of the base.
- LoRA / DoRA:** retention drops 4–8% at the largest ranks.
- AdaLoRA:** keeps retention but FT accuracy collapses to 40–50%.
- Implicit regularization.** Gates stabilize training at large LR / rank.

6 RoBERTa on GLUE

RoBERTa-base on five GLUE tasks, $r \in \{4, 8, 16, 32, 64\}$. Retention = mean MLM perplexity (base ≈ 6.4).

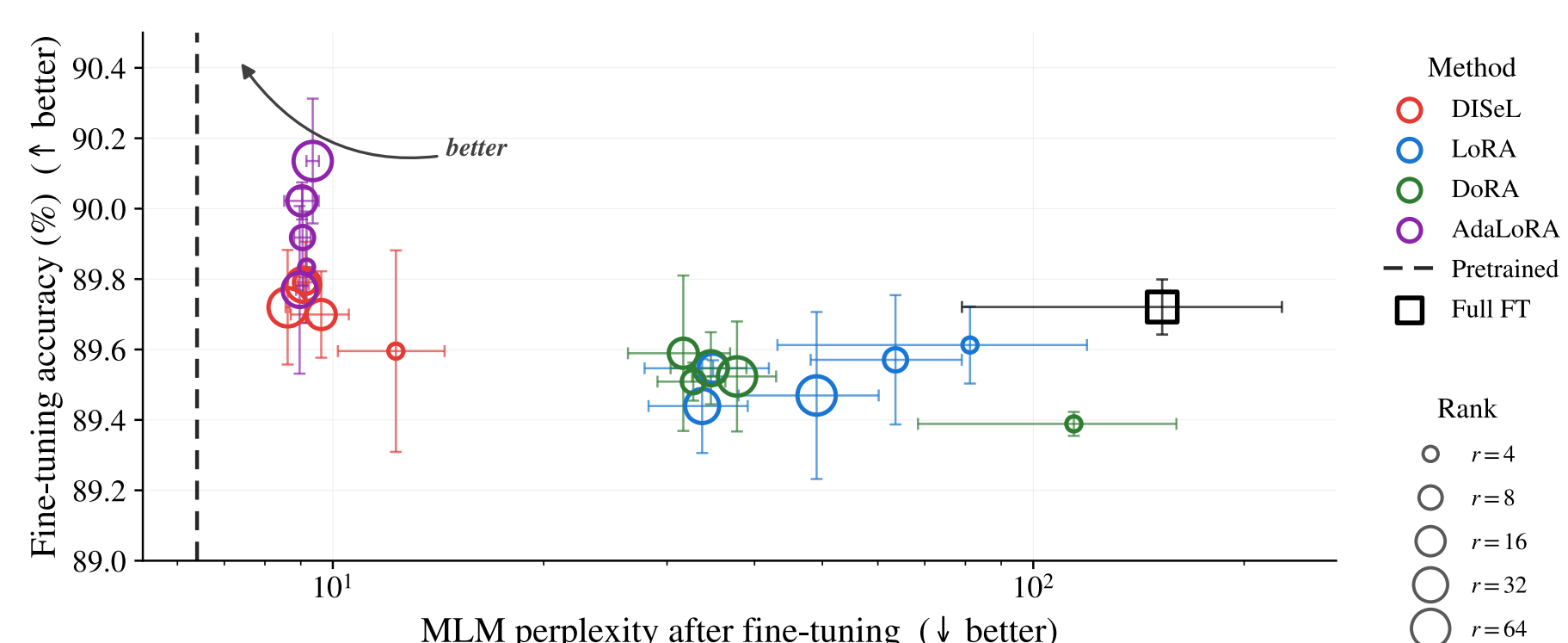


Fig. 3. FT accuracy vs. MLM perplexity. DISeL & AdaLoRA stay near the pre-trained baseline; LoRA / DoRA forget more; Full FT forgets most.

- DISeL & AdaLoRA:** ppl ≈ 9 , matching Full FT accuracy without any pruning schedule.
- Full FT** tops accuracy but pushes ppl to ≈ 150 ; **LoRA / DoRA** sit at ppl 30–100.

7 Retention over training

Retention trajectory of LLaMA-2-7B on MetaMathQA across saved checkpoints, for ranks $r \in \{16, 32, 64, 128, 256\}$.

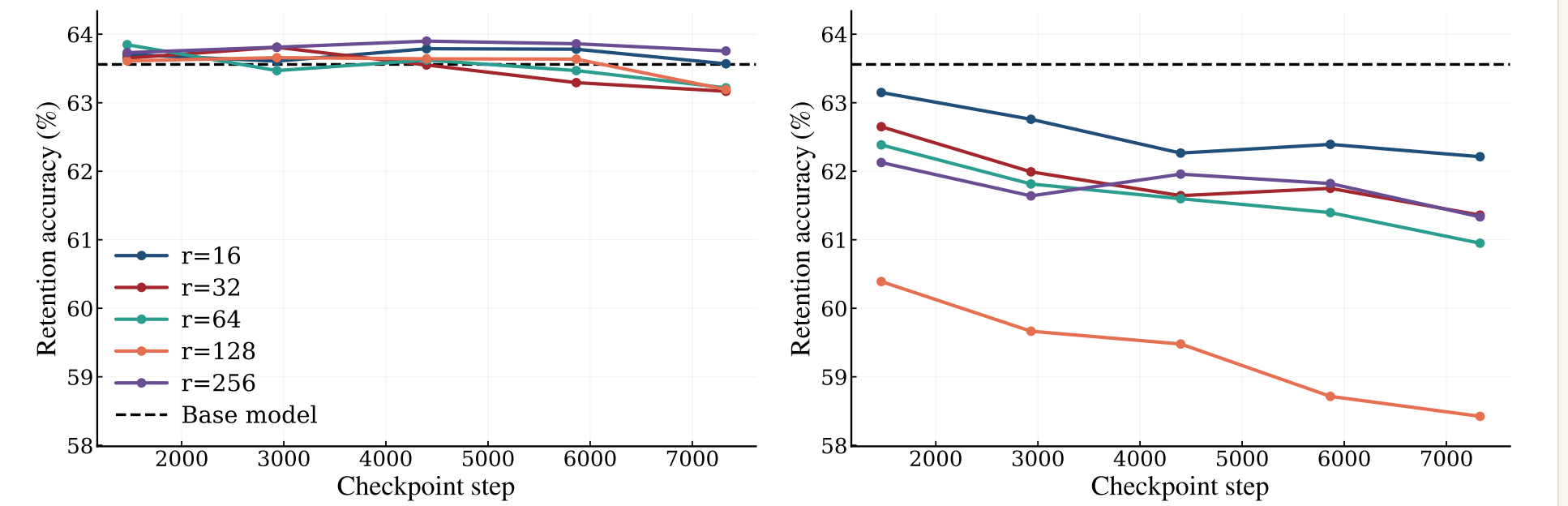


Fig. 6. DISeL (left): near-flat retention at every rank, including the largest. LoRA (right): steady drift downward with both rank and training time.

- DISeL.** Retention within $\approx 0.5\%$ of the base model at every rank and checkpoint.
- LoRA.** Largest drift ($\approx 5\%$) at $r = 128$, not $r = 256$: $r = 256$ needed a smaller learning rate (10^{-4}) for stability while $r = 128$ used a twice-larger step. Rank alone does not decide forgetting — learning rate matters too.
- Mechanism.** DISeL gates stay inactive on pre-training inputs, so neither extra capacity nor larger learning rates leak into drift.

Rank decoupled from forgetting. DISeL safely allows larger r and larger learning rates with no retention penalty.

8 Interpretability: where & when gates open

Held-out samples from three domains (math = FT, general text, code). Gate activations recorded at every adapted layer of the rank-32 LLaMA math adapter.

Domain-conditioned routing

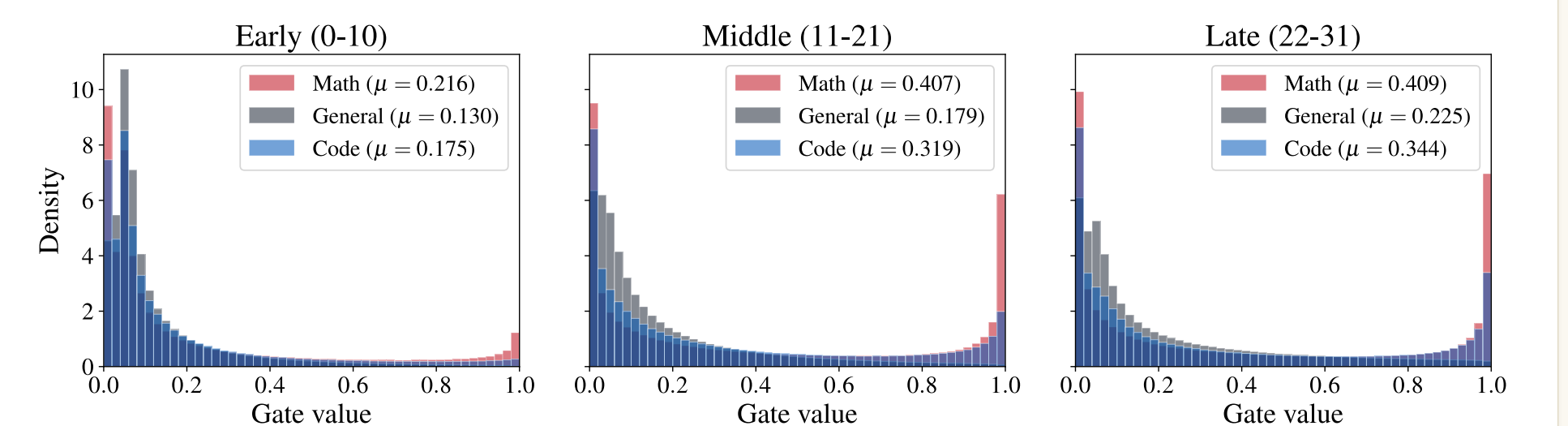


Fig. 4. Early layers stay closed; middle and late layers separate cleanly: math (FT domain) opens gates much wider than general text (pre-training domain). Code, which shares structured reasoning with math, sits in between.

- Early layers** stay closed across all domains, so no adaptation is needed near the input.
- Middle / late layers:** math $\bar{g} \approx 0.41 \rightarrow 0.41$; general text $\bar{g} \approx 0.18 \rightarrow 0.23$, a clear separation.
- Code** sits between math and general text because it shares structured reasoning with math.

Module-level routing

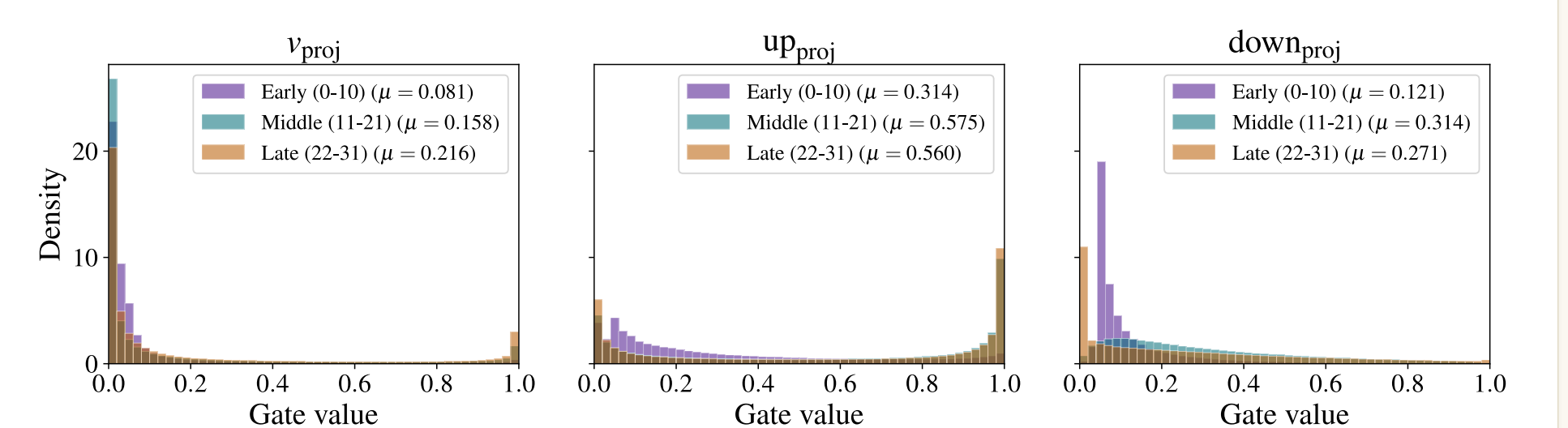


Fig. 5. On math inputs, up_proj in middle/late layers carries the bulk of the adaptation (mean gate ≈ 0.57). v_proj remains comparatively closed.

- up_proj** dominates the math adaptation, with middle/late mean gate ≈ 0.57 .
- down_proj** shows a weaker version of the same trend ($\bar{g} \approx 0.31$).
- v_proj** stays comparatively closed ($\bar{g} \approx 0.16$).
- A free diagnostic.** Gate statistics reveal *where* task-specific capacity concentrates, which is useful for adaptive rank selection and module design.

Takeaways

- Any fixed adapter (full FT, LoRA, DoRA, AdaLoRA) must **compromise** between fitting the task and preserving pre-trained behavior. We prove this in a minimal linear-regression model and show it empirically.
- The Bayes-optimal input-dependent correction is a **sigmoid gate** times the task update, which directly motivates DISeL.
- DISeL adds $rd_x + r$ parameters per block ($\approx 1\%$ of LoRA), **matches LoRA's inference cost**, and starts equal to the pre-trained model.
- Across RoBERTa / GLUE, LLaMA / math, Mistral / code, DISeL **decouples adapter rank from forgetting**: retention stays near the base model while FT accuracy meets or beats LoRA, DoRA, AdaLoRA.
- Learned gate activations give a free **interpretability tool**: they reveal which layers and modules absorb task-specific signal (here: up_proj, middle/late layers).